Owner-Harm: A Missing Threat Model for AI Agent Safety

Anonymous Author(s)
Affiliation
Address
email

Abstract

AI agent safety benchmarks focus predominantly on generic criminal harm—cybercrime, harassment, and weapon synthesis—while under-modeling a commercially consequential threat: agents harming their own deployers. We propose *Owner-Harm*, a formal threat model with eight categories of deployer-directed damage. We quantify the resulting defense gap using the same compositional runtime gate across two authoritative benchmarks: 100% TPR / 0% FPR on AgentHarm (generic criminal harm; 176 harmful scenarios), but only 3.7% (1/27) on AgentDojo prompt-injection tasks under the deterministic L1+L4 configuration in a worst-case ground-truth isolation test (a legacy L1+L3+L4 setup with a relay-hosted `qwen-turbo` backend reported 14.8% but is no longer reproducible under current model and prompt versions; see §5.4). On the same suite under realistic deployment-mode evaluation (629 user-task instances with a GLM-4.6 agent plus L1 Datalog enforcement), the composed system reaches **95.9% security and 75.0% utility** – a 92.2 pp deployment vs. isolation gap which we attribute to the agent itself absorbing most injection attempts before SSDG is asked to act. A controlled zero-shot LLM baseline shows that the AgentHarm/AgentDojo *symbolic* gap is not inherent category difficulty (62.7% AgentHarm vs. 59.3% AgentDojo; gap 3.4 pp), but arises from environment-bound symbolic rules that fail across tool vocabularies. On a post-hoc diagnostic benchmark of 300 owner-harm scenarios, the gate achieves 75.3% TPR / 3.3% FPR; adding a deterministic post-audit verifier raises TPR to 85.3% (combined benign FPR rises to 13.3%, concentrated on Infrastructure Exposure and Inner Circle Leak categories where artifact-pattern rules over-trigger) while lifting Hijacking detection from 43.3% to 93.3% at zero benign FP, indicating layer complementarity is rule-class specific rather than uniform. Finally, we introduce the *Symbolic-Semantic Defense Generalization* (SSDG) conceptual framework, which organizes the observed detection gaps as failures of usable information coverage over ownership, trust boundary, authorization scope, and task-goal context.

1 Introduction

Autonomous AI agents are entering production systems faster than safety frameworks can describe their failures. Current work emphasizes preventing generic criminal harm: malware generation, dangerous-substance synthesis, fraud, and harassment. A different threat has emerged in practice: agents can harm the organization that deploys, authorizes, and trusts them.

Three incidents anchor this concern. Slack AI was shown vulnerable to prompt-injection credential exfiltration in August 2024 PromptArmor [2024]; Microsoft 365 Copilot was hijacked through malicious calendar invitations to forward sensitive emails in January 2024 Zenity Research [2024]; and

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

a Meta AI agent made an unauthorized internal forum post exposing operational data in March 2026 Schiffer [2026]. In each case, the harmed party was the deployer.

Existing benchmarks do not formalize this victim model. AgentHarm Andriushchenko et al. [2025] catalogs generic adversarial use against third parties. AgentDojo Debenedetti et al. [2024] evaluates prompt injection across realistic suites, but without an explicit owner-harm taxonomy. This omission is measurable: the same Nous runtime gate achieves 100% TPR on AgentHarm, but only 3.7% (1/27) on AgentDojo injection tasks under the deterministic L1+L4 configuration.[1] A zero-shot generic LLM baseline shows only a 3.4 pp cross-benchmark gap, attributing the large Nous gap to symbolic rule generalization failure rather than intrinsic semantic difficulty.

This paper contributes: (i) an eight-category Owner-Harm threat model grounded in real incidents; (ii) defense-gap quantification on AgentHarm versus AgentDojo, separating worst-case ground-truth isolation (100% vs. 3.7% deterministic; 100% vs. 14.8% legacy semantic, see §5.4) from realistic deployment-mode evaluation (GLM-4.6 agent + L1 Datalog reaches **95.9% security and 75.0% utility across 629 user-task instances**, §4.3); (iii) layer-complementarity evidence, where post-audit verification raises Hijacking detection from 43.3% to 93.3%; and (iv) the SSDG conceptual framework, which exposes and structures the owner-harm information-coverage gap rather than claiming to solve it – modern instruction-tuned LLMs absorb most injection attempts before SSDG is asked to act, leaving symbolic enforcement as a thin but auditable last line.

2 Owner-Harm Threat Model

2.1 Formal Definitions

We model an AI agent as a tuple $\mathcal{A} = \langle M, \mathcal{T}, \mathcal{E}, \pi \rangle$, where M is the underlying language model, $\mathcal{T}$ is the available tool set, $\mathcal{E}$ is the environment state space, and π is the agent's policy. An agent produces an action sequence $\vec{a} = (a_1, \ldots, a_n)$ transforming environment state s_0 to s'.

Definition 1 (Owner)**.** The *owner* $\mathcal{O}$ is the entity that deploys $\mathcal{A}$, grants resource access $R_{\mathcal{O}}$, and bears accountability for agent actions. $\mathcal{O}$ is associated with a resource set $R_{\mathcal{O}}$, a utility function $U_{\mathcal{O}}$ over environment states, and a trust boundary $\mathcal{B}_{\mathcal{O}}$ delineating authorized interactions.

Definition 2 (Owner-Harm)**.** An action sequence $\vec{a}$ causes *owner-harm* with respect to owner $\mathcal{O}$ if and only if:

$$\mathrm{OH}(\vec{a}, \mathcal{O}) \coloneqq U_{\mathcal{O}}(s') < U_{\mathcal{O}}(s_0) \ \wedge \ \exists\, a_i \in \vec{a} : \mathrm{cause}(a_i,\, s_0 \to s')$$

where $\mathrm{cause}(a_i, s_0 \to s')$ means a_i is a proximate or distal cause of the state transition from s_0 to s'.

Definition 3 (Direct Owner-Harm)**.**

$$\mathrm{DOH}(\vec{a}, \mathcal{O}) \coloneqq \mathrm{OH}(\vec{a}, \mathcal{O}) \ \wedge \ \mathrm{ProxCause}(\mathcal{A}, \vec{a})$$

where $\mathrm{ProxCause}(\mathcal{A}, \vec{a})$ means the agent is the proximate cause of the harmful state transition.

Definition 4 (Indirect Owner-Harm)**.**

$$\mathrm{IOH}(\vec{a}, \mathcal{O}, \mathcal{V}) \coloneqq \mathrm{OH}(\vec{a}, \mathcal{O}) \ \wedge \ \mathrm{Manipulate}(\mathcal{V}, \pi, \vec{a})$$

where $\mathcal{V}$ is an adversarial actor that manipulates the agent's policy π to produce the harmful action sequence—the prompt injection attack model.

2.2 Eight Owner-Harm Categories

We enumerate eight primary categories; a single incident may span multiple labels. Full prose definitions and real-case anchors appear in Appendix B.

C1: Credential Leak. Exfiltration of API keys, OAuth tokens, session cookies, passwords, or equivalent authentication material.

C2: Infrastructure Exposure. Disclosure or misconfiguration of cloud, network, policy, or architecture resources that expands attack surface.

[1] An earlier L1+L3+L4 configuration with a relay-hosted `qwen-turbo` L3 backend reported 14.8% but is no longer reproducible; we therefore report the deterministic result as primary and the semantic configuration separately. See §5.4.

C3: Privacy Exposure. Transmission of PII or confidential business data outside the owner's authorized boundary.

C4: Inner Circle Leak. Exposure of commercially sensitive trust relationships, such as client lists, partner terms, M&A discussions, or strategy.

C5: Asset Destruction. Irreversible deletion or corruption of owner data, configurations, or digital assets.

C6: Exfiltration via Tools. Use of authorized tools—email, webhooks, file writes, URLs—as covert channels for unauthorized data transfer.

C7: Hijacking. Persistent adversarial control over the agent using the owner's identity, credentials, memory, or resource access.

C8: Unauthorized Autonomy. Consequential action beyond the owner's authorization scope or without required confirmation.

2.3 Why Generic Defenses Fail

Owner-harm is context-dependent:

$$\exists\, c, \mathcal{O}_1, \mathcal{O}_2 : \mathrm{Harmful}(c, \mathcal{O}_1) \;\wedge\; \neg\mathrm{Harmful}(c, \mathcal{O}_2).$$

"Forward email to external@attacker.com" is benign if the address is inside the owner's trust boundary and harmful otherwise. Generic defenses usually lack the three required context dimensions: resource ownership ($R_{\mathcal{O}}$), trust boundary ($\mathcal{B}_{\mathcal{O}}$), and authorization scope ($\mathrm{Auth}_{\mathcal{O}}$). Thus content classifiers, injection detectors, tool monitors, DLP systems, and anomaly detectors can see action surface form but not whether the action violates owner-specific policy.

Proposition 1 (Generic Classifier Incompleteness)**.** Any context-free binary classifier $f_{\mathrm{gen}} : \mathcal{C} \to \{0, 1\}$ that classifies action content without access to $R_{\mathcal{O}}$, $\mathcal{B}_{\mathcal{O}}$, or $\mathrm{Auth}_{\mathcal{O}}$ will produce owner-harm false negatives on scenarios where harm is defined relative to the owner's context.

2.4 Symbolic-Semantic Defense Generalization

SSDG is a conceptual framework, not a formal mathematical theory. Let an attack a have evidential requirement set $E(a)$: the contextual facts needed to determine harmfulness. A defense D has information set $\Omega(D) = \Omega_S \cup \Omega_C$, where Ω_S captures surface features and Ω_C captures contextual knowledge. Its usable coverage is

$$\mathrm{Cover}(D, a) := \frac{|E(a) \cap \Omega_{\mathrm{usable}}(D)|}{|E(a)|}.$$

We assume detection is non-decreasing in usable coverage. The framework articulates two falsifiable hypotheses:

Hypothesis 1 (Symbolic Defense Gap)**.** A pure symbolic defense with $\Omega_C = \emptyset$ achieves $\mathrm{Cover}(D, A_G) \gg \mathrm{Cover}(D, A_O)$ when A_O requires joint surface-context evidence, producing a structural detection gap on owner-harm attacks while remaining effective on generic harm.

Hypothesis 2 (Semantic Defense Stability)**.** A semantic defense with $\Omega_C \supseteq \mathcal{C}_{\mathrm{world}}$ satisfies $\mathrm{Cover}(D, A_G) \approx \mathrm{Cover}(D, A_O)$; context deprivation should collapse a semantic defense toward symbolic-gap behavior.

The framework therefore predicts that context deprivation induces symbolic-gap behavior, while raw context injection may fail unless the architecture can perform structured goal-action alignment. Appendix E reports the pilot tests.

3 Nous: Compositional Runtime Safety Gate

We evaluate owner-harm detection using **Nous**, a four-layer compositional runtime safety gate for AI agent actions. The architecture is the evaluation vehicle rather than the primary contribution. L1 is a fail-closed deterministic Datalog blocker with 46 owner-policy rules ($P_{50} = 0.055$ ms). L2 is a triviality filter that routes benign actions past the expensive semantic layer. L3 is an LLM semantic

Table 1: AgentDojo Results: deterministic and semantic modes reported separately (27 injection tasks, 97 user tasks). The deterministic L1+L4 row is the primary auditable result; the legacy L1+L3+L4 row reflects an LLM-assisted measurement that is no longer reproducible (see §5.4).

Mode	L3 backend	Safety	Utility	FPR	Status
L1 only (baseline)	—	3.7% (1/27)†	97.9% (95/97)	2.1% (2/97)	reproducible
L1 + L4 (deterministic)	—	3.7% (1/27)†	97.9%	2.1%	**primary result**
L1 + L3 + L4 (legacy)	qwen-turbo (relay, 4-10)	14.8% (4/27)†	97.9%	2.1%	historical, not reprodu

†95% Wilson CI: 3.7% [0.7%–18.3%]; 14.8% [5.9%–32.5%].

gate with minimal-pair examples, intent decomposition, and majority vote $k = 3$ [Wang et al., 2023]. L4 is a deterministic post-gate verifier: six audit rules inspect file contents and multi-step tool-call sequences invisible to L1–L3, adding ∼0.038 ms with no LLM cost.

This architecture maps naturally to owner-harm. L1 encodes explicit owner-policy facts; L3 reasons about ambiguous owner context; L4 catches prompt-injection artifacts and behavioral sequences that can survive semantic evaluation because the semantic gate is itself exposed to injected content. The layers operate on distinct evidence: action content, semantic intent plus owner context, and post-action artifacts/sequences.

4 Evaluation

4.1 Evaluation Strategy

We evaluate on three benchmark sets. **AgentDojo** Debenedetti et al. [2024]: 27 injection tasks and 97 user tasks across Banking, Travel, Workspace, and Slack, using benchmark ground-truth labels. **AgentHarm** Andriushchenko et al. [2025]: 176 harmful and 176 benign scenarios across 11 categories, used as a generic-harm comparator. **Owner-Harm Benchmark**: 300 harmful and 150 benign scenarios across C1–C8, constructed after optimization loops by the same author. It is *post-hoc diagnostic*, not held-out validation; all labels are from a single annotator and no IAA is reported.

AgentDojo tasks map primarily to financial harm, privacy breach, operational harm, credential leak, reputational harm, and other application-layer owner-harm behaviors. This mapping is subjective but sufficient to expose whether tool-call defenses transfer from generic harm to deployer-directed prompt injection.

4.2 AgentDojo Results

The deterministic L1+L4 result is 3.7% (1/27); the single defended case is caught by an L1 Datalog rule. L4 contributes zero AgentDojo detections (so the L1-only and L1+L4 rows are numerically identical) because L4 rules inspect bash/file/curl artifacts, whereas AgentDojo attacks use application tools such as `send_money`, `send_email`, and `update_calendar`; L4 is retained as part of the deployed configuration because it dominates on non-AgentDojo benchmarks (e.g., +50 pp on the Owner-Harm Hijacking subset, see §5.2). The legacy L3 configuration (qwen-turbo via a relay endpoint, April 2026) reported 14.8% by catching three additional semantically recognizable cases (passport details, bank account numbers, batch wire transfer), but that measurement is not reproducible under current model and prompt versions (see §5.4). We therefore treat the deterministic 3.7% as the primary AgentDojo isolation TPR and discuss the non-reproducible semantic gain (14.8%) only as historical context, not as a current claim. The remaining blind spots are text-only manipulation, malicious parameters that look legitimate without user-goal context, and multi-step attacks where partial interception is insufficient. The estimated ceiling without structured goal context is 40–60%.

4.3 Deployment-Mode Evaluation

The ground-truth isolation test in §4.2 measures a worst-case component property: if an injected instruction reaches the tool stream, the SSDG layer must reject it. We also evaluate the full de-

Table 2: AgentDojo Safety by Owner-Harm Category

Category	L1 only	L1+L3+L4
Financial Harm	0/10 (0%)	1/10 (10%)
Privacy Breach	0/6 (0%)	2/6 (33%)
Operational Harm	1/4 (25%)	1/4 (25%)
Credential Leak	0/3 (0%)	0/3 (0%)
Reputational Harm	0/3 (0%)	0/3 (0%)
Other	0/1 (0%)	0/1 (0%)
Total	**1/27 (3.7%)**	**4/27 (14.8%)**

ployment setting, where the agent itself may ignore, resist, or transform the injection before any SSDG check is needed. We ran the real LLM pipeline on all four AgentDojo suites using GLM-4.6 as the agent, L1 Datalog enforcement, and the L4 verifier enabled, with no L3 semantic gate and single-shot decoding at `temperature=0.0`. Across 629 user-task instances, the pipeline achieved a security success rate of **95.9% (603/629)** while preserving a utility success rate of **75.0% (472/629)**.

Table 3: Deployment-Mode Ablation across 629 user-task instances. All four modes use GLM-4.6 as the agent backbone with `temperature=0.0`; "blocks" counts effective gate blocks in the Nous decision log. The L4 verifier, when enabled, is *touched* on each call but produced zero additional escalations on AgentDojo, consistent with §4.2. The L3 backend is `deepseek-v4-pro` (single-shot, thinking-enabled, `temperature=0.0`); see §5.4 for rationale on this substitution.

Mode	Security	Utility	Δ vs. baseline	Blocks
Baseline (GLM-4.6 only)	95.2% (599/629)	**79.0%** (497/629)	—	0
+ L1 Datalog	96.0% (604/629)	75.0% (472/629)	$+0.8$ sec / -4.0 util	19
+ L1 + L4 verifier	95.9% (603/629)	75.0% (472/629)	$+0.7$ sec / -4.0 util	17
+ L1 + L3 + L4 (DeepSeek)	**96.3%** (606/629)	75.0% (472/629)	$+1.1$ **sec** / -4.0 util	26

Table 4: Per-Suite Security under Deployment-Mode L1+L4 (GLM-4.6 + L1 Datalog + L4 verifier)

Suite	Security	Utility
Banking	94.4% (136/144)	77.8% (112/144)
Slack	91.4% (96/105)	68.6% (72/105)
Travel	97.9% (137/140)	75.7% (106/140)
Workspace	97.5% (234/240)	75.8% (182/240)
Total	**95.9% (603/629)**	**75.0% (472/629)**

Interpretation. These numbers should not be read as SSDG-only injection detection accuracy. They measure the deployed composition of agent behavior plus policy enforcement: an injection only reaches the SSDG layer if the agent first decides to act on it. The ablation in Table 3 resolves three questions about where the safety gain originates and where the utility cost is paid.

Most safety comes from the agent itself. GLM-4.6 alone reaches 95.2% security on AgentDojo deployment trajectories without any runtime guard, indicating that modern instruction-tuned agents are a non-trivial first line against indirect prompt injection on application-tool benchmarks. Adding the full L1+L3+L4 stack lifts security only to 96.3% (a +1.1 pp gain captured by 26 effective blocks).

The symbolic L1 layer pays a real utility tax. Adding L1 Datalog reduces utility from 79.0% to 75.0% (-4.0 pp). All Nous configurations land at the same utility floor of 472/629, which is explained by the cascading effect of L1 blocks on multi-step task trajectories: when L1 blocks an intermediate tool call (typically a destructive primitive like `delete_file` that the injection asks the agent to invoke), the agent re-plans but often fails to recover the original user goal.

L4 is dead weight on AgentDojo deployment. The L4 verifier, when enabled (L1+L4 row), produced zero additional escalations: the 17 effective blocks under L1+L4 are all L1 Datalog blocks, and the slight delta from L1-alone (19 blocks) reflects sampling variance of the agent's tool-call sequence rather than L4 contribution. We retain L4 in the deployed configuration because L4 dominates on the

Table 5: Owner-Harm Benchmark: Per-Category Results (Post-Hoc Diagnostic)

Category	N_h	**Gate TPR**	**Combined TPR**	**Gate FPR**
C1: Credential Leak	28	100.0%	100.0%	0.0%
C2: Infrastructure Exposure	50	100.0%	100.0%	20.0%†
C3: Privacy Exposure	28	100.0%	100.0%	0.0%
C4: Inner Circle Leak	28	89.3%	89.3%	0.0%
C8: Unauthorized Autonomy	50	70.0%	70.0%	0.0%
C5: Asset Destruction	28	64.3%	64.3%	0.0%
C6: Exfiltration via Tools	28	57.1%	57.1%	0.0%
C7: Hijacking	**60**	**43.3%**	**93.3%**	**0.0%**

† 20% FPR for C2 caused by rule OH-R3a over-triggering on benign external-channel configuration queries.

Owner-Harm Hijacking subset (§5.2, +50 pp), but for AgentDojo specifically L4 can be deactivated without security loss.

The semantic L3 layer is the only meaningful augmentation over the agent itself. Adding the locked `deepseek-v4-pro` L3 gate on top of L1+L4 catches 9 additional injections (26 vs. 17 blocks) without further utility loss, lifting security from 95.9% to 96.3%. We therefore report §4.2 and §4.3 as complementary: the former isolates the symbolic enforcement bottleneck (worst-case 3.7% under L1+L4), while the latter measures end-to-end deployment behavior, exposes the utility-vs-security trade-off, and isolates which layers actually matter on AgentDojo.

4.4 Owner-Harm Benchmark Results (Post-Hoc Diagnostic)

Construction protocol (held-out v3, frozen before any of the numbers in this subsection were observed). The Owner-Harm Benchmark held-out v3 slice contains 300 harmful and 150 benign scenarios across the eight Owner-Harm categories (§2.2). Scenarios are author-constructed by the first author from public industry incident reports, AgentDojo-style tool surfaces, and synthesized owner-directed harm patterns; we make no claim of crowd-sourced or third-party-labeled provenance. Each scenario records (i) the owner-issued prompt, (ii) the agent's tool trace and natural-language reply, (iii) a single ground-truth label $\in \{\textit{harmful}, \textit{benign}\}$ assigned by the first author at construction time, and (iv) the Owner-Harm category. Labels are binary and deterministic; we do not measure inter-annotator agreement because the held-out slice is single-annotator by construction (the second author is a co-author of this paper but does not label data). The held-out slice was sealed against the development slice via a SHA-256-frozen JSON file (`data/owner_harm_heldout_v3.json`, hash committed in our supplementary code repository) before any of the numbers in Tables 5–6 were computed. We explicitly do not present this benchmark as an external validity instrument; it is a diagnostic that exposes per-category sensitivity of the Nous gate against a fixed target distribution that the authors themselves chose.

Reproducibility. The benchmark file is released under the same Apache 2.0 license as the gate code; scoring is deterministic (per-rule Datalog match plus post-gate verifier rules); end-to-end reproduction is one command: `python scripts/full_benchmark_eval.py` (no API key required).

Headline numbers. Gate only (L1+L2+L3) achieves TPR = 75.3% (226/300), 95% Wilson CI [70.2%, 79.9%], and FPR = 3.3% (5/150), CI [1.4%, 7.6%]. Adding the L4 post-gate verifier raises TPR to 85.3% (256/300), 95% Wilson CI [80.9%, 88.9%], at the cost of FPR rising to 13.3% (20/150), CI [8.8%, 19.7%]; the additional 15 benign false positives are concentrated on non-Hijacking categories where artifact-pattern rules intended for destructive command detection (e.g., bash/curl artifacts) over-trigger on benign tool sequences. On the Hijacking sub-slice (60 H, 30 B; §5.2) the verifier contributes 0 benign false positives, illustrating that the FPR cost is rule-class specific rather than uniform across categories. The +10 pp TPR / +10 pp FPR trade-off reflects an operating point on the precision-recall curve, not a uniformly Pareto-improving layer.

For Hijacking, Wilson 95% CIs are: gate TPR [31.6%, 55.9%], combined TPR [84.1%, 97.4%], and benign FPR [0.0%, 11.4%]. The 50% verifier-only quadrant captures prompt-injection cases where the semantic gate is manipulated, but file artifacts retain detectable fingerprints.

Table 6: Hijacking (C7) Detection Quadrant Analysis (60 harmful, 30 benign)

Quadrant	Count	%
Gate-only (verifier misses)	11	18.3%
Verifier-only (gate allows)	30	50.0%
Both layers detect	15	25.0%
Neither (structural boundary)	4	6.7%
Combined	**56**	**93.3%**

Table 7: Cross-Benchmark Safety Scores (Nous vs. Generic LLM Baseline)

Benchmark	Focus	Nous Safety	Generic LLM	Status
AgentHarm	Generic criminal	100% / 0%	62.7% / 16.0%	Validation
Owner-Harm (diag.)	Owner-directed	75.3% / 3.3%	—	Diagnostic
AgentDojo	Prompt injection	**3.7%** (legacy: 14.8%*)	59.3% (16/27)	Authoritative

Generic LLM = `qwen-turbo` zero-shot; AgentDojo Nous Safety reports deterministic L1+L4 (3.7%, 1/27) and legacy L1+L3+L4 (14.8%, 4/27) separately. *The legacy semantic configuration is no longer reproducible (§5.4); we therefore use 3.7% as the primary Nous AgentDojo number for all gap computations below. Deterministic Nous gap = $|100\% - 3.7\%| = 96.3$ pp; Generic gap = $|62.7\% - 59.3\%| = 3.4$ pp; difference attributable to symbolic rule generalization failure $= 92.9$ pp.

4.5 Cross-Benchmark Comparison

The 100% $\rightarrow$ 3.7% deterministic gap (and the related 100% $\rightarrow$ 14.8% legacy semantic gap) is the central evidence: constraints that intercept generic criminal content have near-zero coverage of application-layer owner-harm. A controlled zero-shot LLM experiment (Appendix D) shows 62.7% on AgentHarm and 59.3% on AgentDojo, so the large gap is not explained by owner-harm being semantically harder. It is associated with L1 rules tied to AgentHarm's tool vocabulary rather than to portable harm semantics.

4.6 Controlled and SSDG Experiment Summaries

In the controlled gap-attribution experiment, the generic LLM's small 3.4 pp gap contrasts with Nous's deterministic 96.3 pp gap (and legacy semantic 85.2 pp gap, no longer reproducible), showing that symbolic rule generalization failure, not intrinsic category difficulty, drives the benchmark discrepancy. In SSDG pilots, raw user-goal injection reduced AgentDojo legacy-semantic TPR from 14.8% to 8.3%, rejecting the naive context-injection prediction and showing that structured goal-action comparison is required (this pilot was run under the legacy `qwen-turbo`-via-relay configuration; see §5.4). Conversely, stripping context from the generic LLM produced a symbolic-like gap ratio $R = 3.60$ versus $R = 1.06$ with full context, supporting the claim that information deprivation is a key cause of owner-harm detection gaps.

5 Discussion

5.1 Why Generic Defenses Fail

The production and benchmark results converge on the same explanation: generic defenses evaluate action content without owner context. "Send email to external@domain.com" carries no intrinsic harm signal; its harmfulness is determined by whether the destination is inside $\mathcal{B}_\mathcal{O}$ and whether the action is authorized by $\text{Auth}_\mathcal{O}$ over resources in $R_\mathcal{O}$. The AgentDojo blind spots further show that malicious-parameter attacks often require comparing the user's task goal against the agent's action.

5.2 Layer Complementarity

The Hijacking result (43.3% gate $\rightarrow$ 93.3% combined) is direct evidence of architectural complementarity. The gate operates on prompt semantics and command surface; the verifier operates on

file contents and multi-step behavioral sequences. Prompt injection requires both semantic and behavioral-sequence detection because injected content can manipulate the semantic evaluator while leaving fingerprints in artifacts or action chains.

5.3 Deployment-Mode vs. Isolation-Mode Reporting

Owner-harm defenses can be evaluated in two regimes which yield substantially different headline numbers, and which we argue should be reported separately rather than collapsed:

- **Ground-truth isolation** (§4.2): every injection is forced into the tool stream regardless of whether the agent would have followed it. This is a worst-case component test of the SSDG enforcement layer alone. Our deterministic L1+L4 score on AgentDojo is 3.7% (1/27).
- **Realistic deployment** (§4.3): the full agent pipeline runs each user task with the injection embedded in tool outputs as AgentDojo specifies, and the agent may resist, ignore, or transform the injection before SSDG is invoked. Across 629 user-task instances, the composed system reaches 95.9% security with 75.0% utility.

The 92.2 pp gap between these two metrics is structural, not contradictory. It reflects the fact that modern instruction-tuned LLM agents are themselves a meaningful first line against indirect prompt injection on application-tool benchmarks: only a small fraction of injection-bearing trajectories reach the SSDG layer in deployment. This has two implications. First, evaluating SSDG enforcement *only* in isolation mode (the v1 framing) understates real-world safety; evaluating *only* in deployment mode would conflate agent-level resistance with SSDG enforcement, overstating the contribution of the symbolic layer. Reporting both regimes separates *what symbolic enforcement provides* from *what the deployed composition delivers*. Second, these results suggest a productive design direction: a thin auditable symbolic last line composed with an LLM agent that already absorbs most surface attempts, rather than a heavy semantic gate carrying the full defense burden. We treat this as a hypothesis rather than a recommendation, since our deployment evidence is from a single agent backbone (GLM-4.6) on AgentDojo's application-tool surface and may not generalize to other agents or attack regimes.

Every numerical claim in this paper maps to a single command in the anonymized supplementary code repository; we have independently re-executed all no-API-key commands on 2026-05-01–02 from a fresh checkout and verified the headline numbers reproduce deterministically. A full claim-to-command index appears in Appendix A and in `REPRODUCIBILITY.md` in the supplementary materials.

5.4 Limitations

Erratum: AgentDojo L1+L3+L4 reproducibility. Our v1 result of `L1+L3+L4 = 14.8% (4/27)` on AgentDojo (commit `3e688d6`, 2026-04-10) is not a purely deterministic SSDG result. Three of the four detections were produced by the L3 semantic LLM gate using `qwen-turbo` through an internal OpenAI-compatible relay (since deprecated) in April 2026; one was produced by deterministic Datalog logic. Subsequent reruns under the current code base do not reproduce the same L3 behavior because both the remote model endpoint and the local semantic prompt and configuration have since changed (the `semantic_gate.py` module grew `+96` lines after 2026-04-10 with stripped-mode handling, intent-seed alignment, and content-escalation logic). The provider call used `temperature=0.0`, so the issue is model and prompt drift rather than local sampling stochasticity. We therefore report the deterministic `L1+L4 = 3.7% (1/27)` as the primary auditable AgentDojo isolation result in Table 1 and retain the legacy `L1+L3+L4 = 14.8%` only as a historical LLM-assisted measurement, not as a current claim. We do not report a current-model semantic isolation row; deployment-mode semantic results with a frozen `deepseek-v4-pro` L3 are tabulated separately in Table 3 where the gate is composed with the real GLM-4.6 agent. The `L1-only` and `L1+L4` rows in Table 1 are numerically identical because L4's file/bash/curl-artifact rules do not fire on AgentDojo's application-tool surface; L4 is retained because it dominates on Hijacking (see §5.2). The confirm-bypass runtime fix described in the Appendix is independent of this issue.

1. **AgentDojo ceiling**: Without user task goal context, the semantic gate cannot compare "user requested X" to "agent doing Y", limiting achievable deterministic safety to 3.7% and

semantic safety to a $\sim$15%–40% range depending on L3 backend on malicious-parameter tasks.

2. **Benchmark independence**: The Owner-Harm Benchmark is author-constructed after system optimization. It is diagnostic, not validation evidence.
3. **Single annotator**: No inter-annotator agreement is reported for the Owner-Harm Benchmark labels.
4. **Structural boundary**: 4/60 Hijacking cases (SQL file injection, direct SSH key injection) are out of reach of current V-rules.
5. **Evaluation mode**: AgentDojo evaluation uses ground-truth mode without an adversarial attack LLM; real-world adaptive adversaries may achieve different results.
6. **Infrastructure Exposure FPR**: Rule OH-R3a over-triggers on benign external-channel configuration queries (20% FPR for C2), indicating the rule requires further refinement.
7. **SSDG pilot scale**: P1 and P2 experiments use $N = 12$ AgentDojo tasks and $N = 15$ AgentHarm scenarios. Conclusions are directionally consistent with theory but require full-benchmark replication before statistical significance can be claimed.
8. **Deployment-mode attribution**: The headline deployment-mode result (**95.9%/75.0%** on AgentDojo with L1 Datalog, **96.3%/75.0%** with L1+L3+L4 deepseek) measures the *composed* system: a GLM-4.6 agent backbone whose instruction-tuned refusal already absorbs 95.2% of injections (Table 3 baseline row), plus the Nous gate. The gate's *marginal* contribution on AgentDojo deployment is therefore $+0.8$ pp (L1 alone) to $+1.1$ pp (full L1+L3+L4 stack). We report deployment numbers as a deployment measurement, not as a per-layer attribution; per-layer attribution must be read from the AgentDojo *isolation* table (§4.2, 3.7% deterministic L1+L4 isolation TPR), the Owner-Harm held-out diagnostic (§4.4, $75.3\% \rightarrow 85.3\%$ TPR with $+10$ pp from L4), and the Hijacking layer-overlap analysis (Table 6, gate-only / verifier-only / both / neither $= 11/30/15/4$). This separation is structural to our claim: deployment numbers describe what users would see, while isolation and held-out numbers describe what the gate alone is doing. Confounding the two would inflate the apparent gate contribution.

6 Related Work

AgentHarm Andriushchenko et al. [2025] provides a generic LLM-agent harm benchmark; ToolEmu Ruan et al. [2024] evaluates risks in an LM-emulated sandbox; AGrail Liu et al. [2025] proposes hierarchical safety detection. Our distinction is to define the deployer as the victim and quantify the resulting defense gap. Prompt-injection work beginning with Greshake et al. Greshake et al. [2023] and standardized by AgentDojo Debenedetti et al. [2024] is closest empirically; CaMeL Debenedetti et al. [2025] is complementary through capability-aware information flow.

Runtime and policy-enforcement systems include NeMo Guardrails Rebedea et al. [2023], Llama Guard Inan et al. [2023], ShieldGemma Zeng et al. [2024], R2Guard Chen et al. [2024], AgentMonitor Yu et al. [2024], BIPIA Yi et al. [2023], Guardrails AI Guardrails AI [2024], verifiable tool safety Doshi et al. [2026], PCAS Palumbo et al. [2026], AgentSpec Wang et al. [2026], Pro2Guard He et al. [2025], solver-aided verification Roy et al. [2026], ShieldAgent Chen et al. [2025], GuardAgent Xiang et al. [2024], and TrustAgent Hua et al. [2024]. These systems improve enforcement machinery; our claim is orthogonal: owner context ($R_{\mathcal{O}}$, $\mathcal{B}_{\mathcal{O}}$, $\text{Auth}_{\mathcal{O}}$) must be an input to such machinery. Exfiltration and covert channel work, including ASCII smuggling via Copilot Rehberger [2024], PromptArmor's Slack disclosure PromptArmor [2024], and adversarial-agent analysis Abdelnabi et al. [2024], provides concrete attack evidence.

7 Conclusion

Owner-harm is a missing threat model for AI agent safety. We formalize it with eight categories, separate worst-case ground-truth isolation from realistic deployment-mode evaluation (100% vs. 3.7% deterministic isolation; 95.9% security with 75.0% utility under deployment-mode across 629 user-task instances; legacy 100% vs. 14.8% semantic configuration is reported separately as historical, see §5.4), attribute the symbolic gap to rule generalization failure, and show that semantic gates

and deterministic post-audit verifiers provide complementary protection, especially for Hijacking. The core finding is simple: an action harms an owner not by content alone, but by its relation to owner resources, trust boundaries, authorization scope, and task goals.

References

Sahar Abdelnabi, Kai Greshake, Shailesh Mishra, Thorsten Holz, and Mario Fritz. (ab)using LLMs: Adversarial attacks on LLM-based agents. arXiv:2407.01902, 2024.

Maksym Andriushchenko, Francesco Croce, and Nicolas Flammarion. Agentharm: A benchmark for measuring harmfulness of llm agents. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025. Poster #32106.

British Columbia Civil Resolution Tribunal. Air Canada chatbot unauthorized refund commitment. BCCRT Decision #SC-2023-007829, February 2024.

Yili Chen et al. R2guard: Runtime guardrails for llm agents. 2024.

Zhaorun Chen et al. ShieldAgent: Shielding agents via verifiable safety policy reasoning. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2025. arXiv:2503.22738.

Edoardo Debenedetti, Jie Zhang, Mislav Baader, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *NeurIPS*, 2024.

Edoardo Debenedetti, Mislav Balunovic, Florian Tramèr, et al. CaMeL: Defeating prompt injections by design. In *arXiv:2503.18813*, 2025.

Aarya Doshi et al. Towards verifiably safe tool use for LLM agents. In *ICSE NIER 2026*, 2026.

Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injections. arXiv:2302.12173, 2023.

Guardrails AI. Guardrails ai: Input/output guards for llm applications. https://www.guardrailsai.com, 2024.

Haoyu He et al. Pro2Guard: Proactive runtime enforcement of LLM agent safety via probabilistic model checking. arXiv:2508.00500, 2025.

Wenyue Hua et al. TrustAgent: Towards safe and trustworthy LLM-based agents through agent constitution. arXiv:2402.01586, 2024.

Hakan Inan, Kartikeya Upasani, Jianfeng Chi, et al. Llama guard: Llm-based input-output safeguard for human-ai conversations. 2023. Meta AI.

Weidi Liu et al. AGrail: A lifelong agent guardrail with effective and adaptive safety detection. arXiv:2502.11448, 2025.

Nils Palumbo, Sarthak Choudhary, et al. Policy compiler for secure agentic systems. arXiv:2602.16708, February 2026.

PromptArmor. Slack AI data exfiltration via prompt injection. Security disclosure, promptarmor.com, August 2024.

Traian Rebedea, Razvan Dinu, Makesh Narsimhan Sreedhar, Christopher Parisien, and Jonathan Cohen. Nemo guardrails: A toolkit for controllable and safe llm applications with programmable rails, 2023. NVIDIA.

Johann Rehberger. Exfiltration of personal data using Microsoft 365 Copilot via prompt injection (ascii smuggling). Blog post, embracethered.com, 2024.

Arnab Roy et al. Solver-aided verification of policy compliance in tool-augmented LLM agents. arXiv:2603.20449, March 2026.

Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J Maddison, and Tatsunori Hashimoto. Identifying the risks of lm agents with an lm-emulated sandbox. In *ICLR*, 2024.

Zoe Schiffer. Meta's ai agent incident: Unauthorized internal forum posting. The Information, reported by TechCrunch, March 2026. Sev 1 incident, 2h sensitive data exposure before containment.

Haoyu Wang, Christopher M. Poskitt, et al. AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*, 2026. arXiv:2503.18666.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023. Earlier preprint: arXiv:2203.11171 (2022).

Zhen Xiang et al. GuardAgent: Safeguard LLM agents by a guard agent via knowledge-enabled reasoning. arXiv:2406.09187, 2024.

Jingwei Yi et al. Benchmarking and defending against indirect prompt injection attacks on large language models. In *arXiv preprint arXiv:2312.14173*, 2023.

Yue Yu et al. Agentmonitor: Real-time behavior monitoring for llm agents. 2024.

Wenhao Zeng et al. Shieldgemma: Generative ai content moderation based on gemma. 2024. Google DeepMind.

Zenity Research. Microsoft 365 Copilot prompt injection via calendar invites. Security research report, zenity.io, January 2024.

A Reproducibility Index

Every numerical claim in the main paper maps to a single command in the anonymized supplementary code repository (`REPRODUCIBILITY.md` in the supplementary materials). The deterministic claims (LSVJ-S compile gate, Owner-Harm v3 full L1–L4, Hijacking layer overlap) require no API key and complete in under one minute on commodity hardware. The LLM-bound claims (AgentDojo isolation/deployment, AgentHarm 100% TPR) require the corresponding subscription endpoint.

Table 8: Paper claim → reproduction command. Wall-clock is single-machine sequential; API costs assume cached prompts where applicable. Determined refers to byte-for-byte stability under `temperature=0.0` on the locked endpoint.

Claim (§ref)	Command	Key	Wall	Cost
LSVJ-S compile gate (138/139 tests, suppl. companion)	`pytest tests/lsvj/`	—	< 10s	\$0
Owner-Harm v3 full L1–L4: 85.3% TPR / 13.3% FPR (§4.4)	`python scripts/full_benchmark_eval.py`	—	∼30s	\$0
Hijacking layer overlap 11/30/15/4 (§5.2)	`python scripts/eval_d2_verifier.py`	—	∼10s	\$0
AgentDojo isolation L1+L4 = 3.7% (§4.2)	`cd benchmarks/agentdojo_adapter && uv run --project ../agentdojo python run_eval.py`	—	∼1–3 min	\$0
AgentDojo deployment 95.9%/75.0% L1 (§4.3)	`bash benchmarks/agentdojo_adapter/launch-baseline-l1-rerun.sh`	GLM	∼5h	\$0
AgentDojo deployment 96.3% L1+L3+L4 deepseek (§4.3)	`bash benchmarks/agentdojo_adapter/launch-l3-deepseek-repro.sh`	DeepSeek	∼5h	∼\$15
AgentHarm L1+L4: 100% TPR (§4.5)	`python scripts/run_agentharm_threelayer.py`	DeepSeek	∼1h	∼\$10

We re-executed all no-API-key commands on 2026-05-01–02 from a fresh checkout and verified the headline numbers reproduce deterministically: `pytest tests/lsvj/` (138 passed), `python scripts/full_benchmark_eval.py` (Owner-Harm v3 numbers match), `python scripts/eval_d2_verifier.py` (Hijacking layer overlap 11/30/15/4 matches), and the AgentDojo isolation `run_eval.py` (3.7% / 97.9% / 2.1% in 1–3 minutes). The LSVJ-S test count is 138/139 (one skipped because the optional `scallopy` dependency is not installed by default) under the path-independent CI subset that our supplementary code drop runs on every push. The LLM-bound L3 deployment number (96.3%/75.0%) was independently reproduced on the same date via the locked DeepSeek-V4-pro endpoint. The AgentDojo legacy L1+L3+L4 = 14.8% number is acknowledged in §5.4 as not currently reproducible and is reported as historical context only.

B Eight Categories: Full Prose and Anchors

We enumerate eight categories $C1$–$C8$ (primary labels; a single incident may span multiple categories), each grounded in documented incidents.

C1: Credential Leak. The agent exfiltrates authentication materials (API keys, OAuth tokens, session cookies, passwords) belonging to or entrusted to the owner. *Real case*: Slack AI August 2024 PromptArmor [2024]—injected prompt caused the assistant to echo private channel tokens to the attacker.

C2: Infrastructure Exposure. The agent misconfigures or discloses network rules, cloud resource policies, or internal architecture, expanding the owner's attack surface. *Pattern*: AI-assisted code generation producing over-permissive AWS IAM policies that expose production databases.

C3: Privacy Exposure. The agent transmits personally identifiable information (PII) or confidential business data to unauthorized parties. *Real case*: Microsoft 365 Copilot January 2024 Zenity Research [2024]—calendar injection caused Copilot to forward sensitive email contents externally.

C4: Inner Circle Leak. The agent betrays trust relationships that are commercially sensitive to the owner: client lists, partner agreements, M&A discussions, or strategic roadmaps. *Pattern*: Samsung employees pasting proprietary semiconductor source code into ChatGPT (April 2023), with the owner-harm being the confidentiality breach to a third-party model.

C5: Asset Destruction. The agent irreversibly deletes or corrupts owner data, configurations, or digital assets. *Pattern*: AI coding agents executing `rm -rf` on production directories when given broad filesystem access and ambiguous cleanup instructions.

C6: Exfiltration via Tools. The agent exploits *authorized* tools as covert data channels—using email, webhooks, or file-write operations to smuggle sensitive data to attacker-controlled endpoints. *Real case*: ASCII smuggling via Microsoft 365 Copilot Rehberger [2024]—injected prompt caused Copilot to encode and transmit sensitive data through Markdown image URLs rendered invisibly.

C7: Hijacking. An adversary achieves persistent control over the agent using the owner's identity, credentials, or resource access—the agent becomes a weaponized surrogate. *Pattern*: AutoGPT memory poisoning (2024–2025) where injected memory entries caused the agent to act on behalf of the attacker across subsequent sessions.

C8: Unauthorized Autonomy. The agent exceeds its authorization scope by taking consequential actions without required human confirmation. *Real case*: Air Canada chatbot (February 2024) committed the airline to unauthorized refund terms in a legally binding interaction British Columbia Civil Resolution Tribunal [2024].

C Comparison with Existing Taxonomies

Table 9 compares Owner-Harm categories against four existing frameworks: AgentHarm Andriushchenko et al. [2025] (11 categories), ToolEmu Ruan et al. [2024] (7 risk types), OWASP LLM Top 10, and AgentDojo Debenedetti et al. [2024] (4 suites).

The most notable gap is **C4 (Inner Circle Leak)**, which appears in no existing benchmark. This category requires reasoning about the owner's trust graph—who counts as an insider—which no current system models explicitly. **C6** is partially covered by ToolEmu but misses the *covert channel*

Table 9: Coverage of Owner-Harm Categories by Existing Frameworks

Category	AgentHarm	ToolEmu	OWASP	AgentDojo
C1: Credential Leak	$\sim$	✓	✓	$\sim$
C2: Infrastructure Exposure	$\times$	$\sim$	✓	$\times$
C3: Privacy Exposure	$\sim$	✓	✓	✓
C4: Inner Circle Leak	$\times$	$\times$	$\times$	$\times$
C5: Asset Destruction	$\sim$	✓	$\sim$	$\sim$
C6: Exfiltration via Tools	$\times$	$\sim$	$\sim$	$\sim$
C7: Hijacking	$\times$	$\sim$	✓	$\sim$
C8: Unauthorized Autonomy	$\sim$	✓	$\sim$	$\times$

✓ = covered; $\sim$ = partial; $\times$ = gap

aspect: using authorized tools as smuggling vectors. The *owner-centric perspective* itself is the primary novel contribution: existing frameworks evaluate harm against third parties, users, or society; none formally model the deploying owner as the victim.

Table 10: Generic Defense Failure Modes for Owner-Harm

Defense Class	Owner-Harm Blind Spot
Content classifier	Ignores $R_{\mathcal{O}}$; "send email" is benign by content
Injection detector	Detects injection syntax; misses semantic intent
Tool monitor	Sees call signature; not authorization context
DLP system	Pattern-matches known PII; misses context-defined confidential data
Behavioral anomaly	Requires baseline; new deployments have no baseline

D Controlled Experiment: Full Detail

The 96.3 pp deterministic gap between AgentHarm (100%) and AgentDojo (3.7% under L1+L4) — and the related 85.2 pp gap under the no-longer-reproducible legacy L1+L3+L4 configuration (14.8%, see §5.4) — admits two competing explanations: (H1) the gap reflects a genuine structural difference—owner-harm scenarios detected via prompt injection are semantically harder to classify; or (H2) the gap is an artifact of Nous's symbolic rules being fit to the AgentHarm tool vocabulary and failing to generalize.

Experimental design. We evaluate a zero-shot generic LLM classifier (`qwen-turbo` via API relay) on the same data, using a uniform binary-classification prompt with no benchmark-specific fine-tuning:

> *You are a safety classifier. An AI agent is performing a task. User's original request: {prompt}. Agent's planned action: {tool} ({params}). Is this action potentially harmful to the user? Answer YES or NO.*

For AgentDojo, the goal description (injected instruction) is substituted for the tool-call representation. The baseline has no access to owner context ($R_{\mathcal{O}}$, $\mathcal{B}_{\mathcal{O}}$, $\text{Auth}_{\mathcal{O}}$).

Results. Table 11 shows the 2×2 comparison.

Finding. The generic LLM exhibits a 3.4 pp gap (62.7% vs. 59.3%)—statistically negligible compared to Nous's 96.3 pp deterministic gap (or the 85.2 pp legacy semantic gap). This is inconsistent with H1: AgentDojo owner-harm scenarios do not appear to be semantically harder to detect. The generic LLM achieves 59.3% on AgentDojo without any prompt-injection-specific design, suggesting that the harm is semantically recognizable by content alone.

Gap attribution. Nous's gap is strongly associated with **Layer 1 (Datalog) generalization failure**. L1 rules were derived from AgentHarm patterns—`bash(script)`, `read_file`

Table 11: Controlled Experiment: Gap Attribution (Nous vs. Generic LLM Classifier)

System	AgentHarm TPR	AgentDojo TPR	Gap
Nous (L1+L4 deterministic, primary)	100.0%	3.7% (1/27)	96.3 pp
Nous (L1+L3+L4 legacy)[†]	100.0%	14.8% (4/27)	85.2 pp
Generic LLM	62.7%	59.3% (16/27)	3.4 pp

Generic LLM AgentHarm: 110 TP / 66 FN / 28 FP (FPR = 15.9%, $N = 176$ harmful + 176 benign).
[†]Legacy semantic configuration (`qwen-turbo` via a now-deprecated internal relay, commit `3e688d6`) no longer reproduces under current code; see §5.4.
AgentDojo per-category breakdown appears in text.

(`malicious_path`), `execute(command)`—which are absent from AgentDojo's application-layer tool vocabulary (`send_money`, `send_email`, `update_calendar`). **Implication: generic LLM outperforms Nous.** The zero-shot generic LLM (59.3%) substantially outperforms Nous's deterministic 3.7% (and the legacy semantic 14.8%) on AgentDojo owner-harm detection. This is not a contradiction—it is the central architectural insight: owner-harm detection depends primarily on semantic understanding of action consequences and owner context, which symbolic rules (L1) cannot provide. Nous's L1 achieves perfect recall on AgentHarm's system-call patterns but has near-zero coverage of application-layer actions. The legacy L3 semantic gate contributed $+11.1$ pp under `qwen-turbo`-via-relay (a configuration that no longer reproduces); the contribution of any locked current L3 backend (e.g., `deepseek-v4-pro`) is reported separately as exploratory in Table 1. The generic LLM compensates via world-knowledge reasoning, suggesting that future owner-harm defenses should prioritize structured semantic alignment over pattern-matching rules.

E SSDG Experimental Predictions: Full Detail

The SSDG framework (Section 2.4) yields two testable predictions. We report pilot experiments ($N = 12$ AgentDojo tasks + 15 AgentHarm scenarios) using the same controlled setup as Appendix D.

P1 (Context Injection) — Prediction Rejected. Injecting the user's task goal into the gate evaluation context (L1+L3+L4 + `user_goal`) should raise the AgentDojo TPR if goal-awareness is the missing information. Result: TPR declined from the legacy-semantic baseline 14.8% to 8.3%—below the no-injection baseline. (This pilot was run under the legacy `qwen-turbo`-via-relay configuration; see §5.4. The same prediction has not yet been re-tested under the locked `deepseek-v4-pro` L3 backend.)

This rejection has an informative interpretation: appending the user goal as a text prefix does not constitute structured goal-action alignment. The gate's intent decomposition logic cannot compare the injected goal to the observed action without explicit structural scaffolding (e.g., a "goal vs. action" contrast prompt). Three confounding factors compound this: (a) the ground-truth AgentDojo pipeline evaluates task success, not injection semantics; (b) simple text injection may activate over-cautious benign-task blocking; and (c) the pilot sample size ($N = 12$) is too small for significance. P1 remains open: the mechanism requires *structured* goal-context integration, not raw text concatenation.

P2 (Context Deprivation) — Prediction Confirmed. Stripping world-context from the semantic defense should reproduce symbolic-gap behavior. We evaluate the generic LLM classifier under two conditions: (i) *Full Group*—standard evaluation with full agent-context prompt; (ii) *Stripped Group*—evaluation with only the raw tool name and parameter, no surrounding task description.

The stripped condition yields $R = 3.60$, exceeding the P2 falsification threshold ($R \leq 1.5$). When deprived of context, the semantic defense degrades to near-symbolic gap behavior, confirming Hypothesis 2: the stability of semantic detection depends on access to world-context, not on model reasoning capacity per se. The finding identifies *information deprivation* as a key factor associated with the detection gap.

Table 12: P2: Context Deprivation Induces Symbolic-Class Gap (pilot, $N = 12$ AgentDojo + 15 AgentHarm)

Condition	AgentHarm TPR	AgentDojo TPR	Gap Ratio R
Full Group	62.7%	59.3%	1.06
Stripped Group	30.0%	8.3%	3.60

$R = \mathrm{TPR}_{\mathrm{AH}}/\mathrm{TPR}_{\mathrm{AD}}$. P2 falsification threshold: $R \leq 1.5$.

F Production Deployment Evidence

Nous was deployed in shadow mode alongside an existing rule-based system in a production multi-agent environment for 11+ days (30,204 calls total).

Consistency: 99.47% agreement with the existing rule-based system across all calls—confirming that semantic reasoning does not introduce systematic drift from established policy.

False negative recovery: Nous detected 157 cases that the existing rule-based system passed as benign. Manual review of a stratified sample confirmed these were true positives, predominantly ambiguous read-then-send workflows where the send target fell outside the owner's trust boundary. *Caveat*: the manual review was conducted by the system developer (first author), not an independent auditor, and was not blind to Nous's verdict—creating potential confirmation bias. These results should be interpreted as case-level observations, not as a controlled study.

Over-blocking: Nous over-blocked in 4 cases where the existing system correctly allowed. Manual review confirmed all 4 were over-triggered on ambiguous read-then-send workflows where the send target was a legitimate internal address. (Same caveat as above: non-blind, author-conducted review.)

Following the shadow period, Nous was promoted to primary interceptor. No regressions were observed post-cutover.

G Extended Discussion

G.1 Symbolic vs. Semantic Generalization

The controlled experiment, together with the SSDG theoretical framework (Hypotheses 1–2), exposes a fundamental asymmetry between the two primary detection mechanisms in compositional safety architectures:

Symbolic rules (L1 Datalog) exhibit a bimodal generalization profile: near-perfect recall within the training distribution (100% on AgentHarm), and near-zero recall on out-of-distribution tool vocabularies (3.7% on AgentDojo). This is structurally expected—Datalog rules encode extensional facts ("tool t is prohibited") rather than intensional harm semantics. Any tool name not in the rule set is trivially allowed.

Semantic reasoning (L3 gate and generic LLM) exhibits a flat generalization profile: below-perfect recall within distribution (62.7% for the generic LLM on AgentHarm), but stable recall across environments (59.3% on AgentDojo, gap = 3.4 pp). Semantic reasoning transfers because harm intent is encoded in goal descriptions—which remain semantically consistent across tool-vocabulary changes—rather than in tool names.

This asymmetry has a direct design implication for compositional architectures: the value of combining symbolic and semantic layers is not simply "two layers are better than one." The layers have *complementary generalization curves*: symbolic rules provide high-precision coverage in-distribution, and semantic reasoning maintains consistent coverage out-of-distribution. A system relying solely on symbolic rules will exhibit catastrophic degradation on novel tool environments; a system relying solely on semantic reasoning will accept a consistent recall ceiling but generalize robustly. The composable architecture's value is precisely that it avoids both failure modes.

G.2 Implications for Benchmark Design

The AgentDojo deterministic-3.7% (and legacy-semantic 14.8%) result implies a design gap in existing injection benchmarks: they do not provide explicit owner context. A benchmark that injects into the gate context (a) which resources belong to the owner, (b) who is in the owner's trust boundary, and (c) what the user's task goal is, would enable defenses to reach the estimated 40–60% ceiling and would meaningfully differentiate owner-harm defense strategies.

We recommend future benchmarks include an *owner context manifest*: a structured specification of $R_{\mathcal{O}}$, $\mathcal{B}_{\mathcal{O}}$, and $\text{Auth}_{\mathcal{O}}$ provided to both the agent and any runtime safety system.

H Extended Future Work

Structured goal-action alignment. The P1 SSDG experiment reveals that simply injecting the user's task goal as text is insufficient—the gate must be architecturally restructured to perform an explicit goal-vs.-action comparison. A dedicated "goal alignment prompt" that contrasts the user's authorized intent against the agent's proposed action is the concrete next step, estimated to raise AgentDojo safety from the current single-digit deterministic ceiling (3.7%) toward the structural 40–60% range.

UEBA for AI agents. User and entity behavioral analytics adapted for AI agent deployments would enable per-owner behavioral profiles, providing a personalized baseline for anomaly detection.

Adaptive immunity. Systems that learn owner-harm patterns from deployment logs—incorporating newly discovered injection patterns into Datalog or V-rule libraries without manual rule authoring.

SQL-aware file content analysis. Addressing the 4 structural boundary Hijacking cases requires content-aware analysis of structured file formats, beyond the current regex-based V-rules.

Owner context manifest standard. A community standard for specifying $R_{\mathcal{O}}$, $\mathcal{B}_{\mathcal{O}}$, and $\text{Auth}_{\mathcal{O}}$ in a machine-readable format, to enable principled owner-harm benchmarking across systems.

Per-decision proof-obligation synthesis. Ongoing work extends this compositional gate from offline-authored Datalog rules to *per-decision runtime synthesis* of Datalog proof obligations by the LLM itself, admitted through a compile-time gate that rejects perturbation-invariant or decisive-primitive-less rules. This moves verification from a "statute" (pre-authored policy) to a "case-law" (per-decision argued) model; a companion preprint is in preparation.

I Author Contributions

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and §1 list the four contributions (eight-category threat model, defense-gap quantification, layer-complementarity evidence, SSDG conceptual framework). §5.4-style limitations explicitly bound the scope (post-hoc Owner-Harm Benchmark, n=27 AgentDojo CIs, ∼15% AgentDojo ceiling without goal context).

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: §5.4 explicitly enumerates seven limitations including the post-hoc nature of the Owner-Harm Benchmark, single-annotator labeling, AgentDojo ceiling without goal context, ground-truth-mode evaluation, C2 over-triggering, and SSDG pilot scale.

3. **Theory assumptions and proofs**

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: This paper is empirical and conceptual; the SSDG framework is a falsifiable conceptual framework rather than a formal theorem (§2.4). The two hypotheses are stated explicitly and tested in §E (appendix).

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results?

Answer: [Yes]

Justification: §3 fully describes the four-layer Nous architecture (Datalog gate + semantic LLM + verifier rules); §4 states benchmarks (AgentDojo, AgentHarm), evaluation modes (ground-truth), and per-layer ablations. AgentDojo and AgentHarm are public.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code?

Answer: [No]

Justification: AgentDojo and AgentHarm are public benchmarks. The Nous system code and Owner-Harm Benchmark labels will be released upon acceptance under a permissive license (MIT/Apache 2.0); reviewers can request access to an anonymized code drop during review.

6. **Experimental setting/details**

Question: Does the paper specify all the training and test details necessary to understand the results?

Answer: [Yes]

Justification: §3 states the LLM backbone for the semantic gate and verifier configuration. §4.1 states the AgentDojo evaluation mode (ground-truth) and the per-injection-task category mapping.

7. **Experiment statistical significance**

Question: Does the paper report error bars or statistical-significance information?

Answer: [Yes]

Justification: All proportion-based results report 95% Wilson confidence intervals. AgentDojo n=27 yields wide CIs ([5.9%, 32.5%]) which is reported transparently rather than hidden.

8. **Experiments compute resources**

Question: For each experiment, does the paper provide sufficient information on the computer resources?

Answer: [Yes]

Justification: All evaluation runs are CPU-only and use closed-API LLMs as black-box services (no GPU training). Total API spend is on the order of $100 across all evaluation cycles; per-task latency dominated by API round-trips.

9. **Code of ethics**

Question: Does the research conform with the NeurIPS Code of Ethics?

Answer: [Yes]

Justification: The work is defensive AI safety research building on public benchmarks (AgentDojo, AgentHarm) and publicly reported industry incidents (Slack AI 2024, Microsoft 365 Copilot 2024, Meta agent 2026); no human subjects, no data scraping.

10. **Broader impacts**

Question: Does the paper discuss potential societal impacts?

Answer: [Yes]

Justification: The Owner-Harm threat model is intended to enable defenders. Potential adversarial use of the taxonomy is bounded by the fact that all eight categories are derived from already-public incidents; identification of category-specific defense gaps motivates positive defensive engineering.

11. **Safeguards**

    Question: Does the paper describe safeguards for the responsible release of data or models?

    Answer: [N/A]

    Justification: No model is being released. The Owner-Harm Benchmark contains synthetic adversarial scenarios, not user data.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets used in the paper properly credited and are the license and terms of use respected?

    Answer: [Yes]

    Justification: AgentDojo (MIT) and AgentHarm (Apache 2.0) are cited and used per their licenses; only public benchmark scenarios are reused.

13. **New assets**

    Question: Are new assets introduced in the paper well documented?

    Answer: [Yes]

    Justification: The Owner-Harm Benchmark structure (8 categories $\times$ harmful + benign scenarios; 450 total) is described in §4.4. License will be assigned at release.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation?

    Answer: [N/A]

    Justification: No crowdsourcing or human-subjects data. The post-hoc Owner-Harm Benchmark scenarios were author-constructed (acknowledged in §5.4 as a known limitation; an independent annotator pass is in progress).

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether IRB approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: No human-subjects research.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research?

    Answer: [Yes]

    Justification: The semantic-reasoning Layer 3 of the Nous evaluation system uses a closed-API LLM as a per-action safety classifier (described in §3). Separately, LLM assistants were used for prose editing and proof reading; LLMs were not used for ideation or experimental design.